

FLOWDAR

Détection automatique des inondations et guidage citoyen à Douala

Document technique — Spécifications Backend

Angular Talent Lab 2026 — Orange Digital Center Douala

1. Contexte et problème résolu

Près de 48% de la ville de Douala est exposée aux risques d'inondation. Entre 2024 et 2025, plus d'un million de Camerounais ont été affectés. Lorsqu'il pleut fort, aucun outil numérique ne permet aux habitants de savoir en temps réel quelles rues sont praticables.

Les solutions existantes présentent des lacunes majeures :

- ONACC — prévisions régionales décennales (tous les 10 jours), web uniquement, pas de signalement citoyen
- Facebook / WhatsApp — informations non géolocalisées, non structurées, retrouvées trop tard
- Croix-Rouge / OCHA — réponse post-catastrophe uniquement

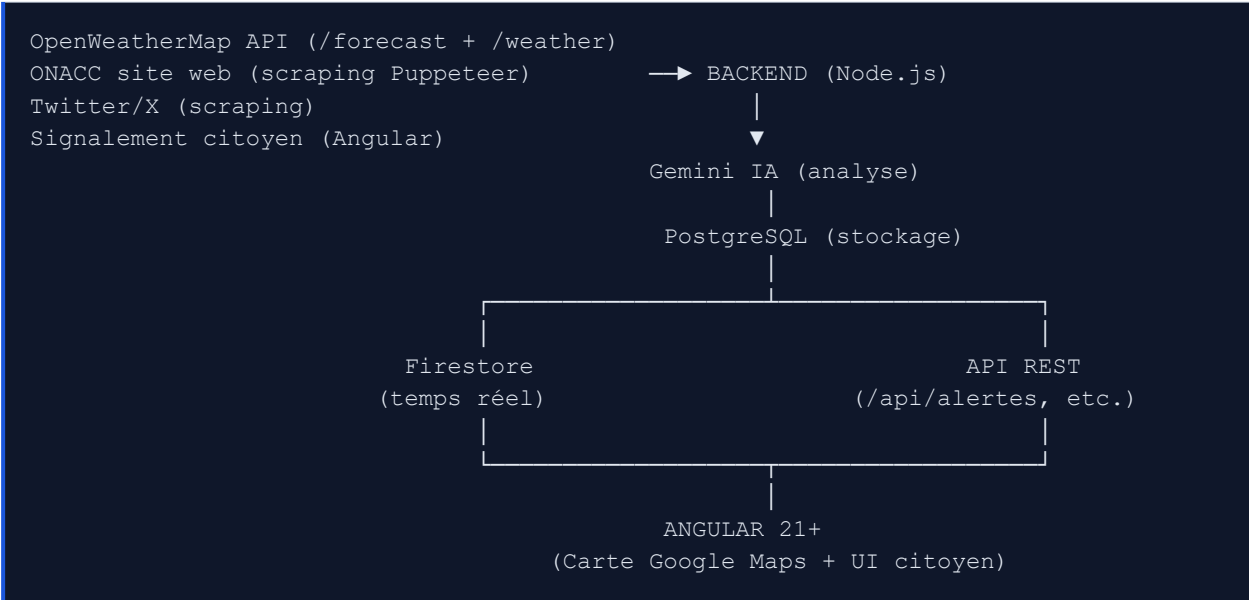
Flowdar comble ce vide : détection automatique via météo + signalement citoyen + guidage intelligent vers des itinéraires sûrs.

2. Architecture globale du système

Le système repose sur trois grandes couches qui communiquent entre elles :

Couche	Technologie	Rôle
Sources de données	OpenWeatherMap, ONACC (scraping), Twitter/X (scraping), Signalements citoyens	Alimenter le système en données brutes
Backend	Node.js + Express, PostgreSQL, Gemini API, Puppeteer/Cheerio	Collecter, analyser, stocker, exposer les données
Frontend	Angular 21+, TailwindCSS, Google Maps API, Firebase	Afficher les alertes, recueillir les signalements citoyens

Flux de données global :



3. Stack technique recommandée

Composant	Technologie	Justification
Runtime	Node.js 20+ LTS	Léger, performant pour APIs REST et scraping asynchrone
Framework API	Express.js	Simple, bien documenté, parfait pour exposer des endpoints REST
Base de données	PostgreSQL 16+	Requêtes géospatiales (PostGIS), historique complexe, fiabilité
Extension géo	PostGIS	Stockage et requêtes sur les coordonnées GPS des zones à risque
Scraping web	Puppeteer + Cheerio	Puppeteer pour les sites dynamiques (JS), Cheerio pour HTML statique
IA	Gemini API (google/generative-ai)	Analyse des signalements texte, seuils intelligents, résumés
Tâches planifiées	node-cron	Jobs automatiques toutes les 15-30 min (météo, scraping)
Déploiement	Railway ou Render	Gratuit pour projets de formation, déploiement simple
Stockage fichiers	Firebase Storage	Photos des signalements citoyens (déjà dans la stack Angular)

4. Endpoints API à exposer à Angular

Le backend expose une API REST que le frontend Angular consomme. Voici tous les endpoints nécessaires :

4.1 Alertes

Métho de	Endpoint	Description	Réponse
GET	/api/alertes	Toutes les alertes actives	Array d'alertes avec coordonnées GPS et niveau
GET	/api/alertes/:id	Détail d'une alerte	Objet alerte complet avec confirmations
POST	/api/alertes/signaler	Signalement citoyen manuel	Alerte créée, passée à Gemini pour analyse
POST	/api/alertes/:id/confirmer	Confirmer une alerte existante	nb_confirmations +1
POST	/api/alertes/:id/resoudre	Marquer une alerte résolue	Statut alerte → résolu
GET	/api/alertes/historique/:quartier	Historique par quartier	Array d'alertes passées triées par date

4.2 Zones à risque

Métho de	Endpoint	Description	Réponse
GET	/api/zones-a-risque	Liste de toutes les zones inondables	Array avec nom, coordonnées GPS, niveau risque de base
GET	/api/zones-a-risque/:id	Détail d'une zone	Zone + historique des alertes associées

4.3 Itinéraire sûr

Métho de	Endpoint	Description	Réponse
POST	/api/itineraire	Calcule un itinéraire évitant les zones alertées	Liste des zones actives à éviter, passée à Google Maps côté Angular

Note : Le calcul de l'itinéraire est fait côté Angular via Google Maps Directions API. Le backend fournit uniquement la liste des coordonnées GPS des zones actives à éviter.

4.4 Météo

Métho de	Endpoint	Description	Réponse
GET	/api/meteo/actuelle	Météo actuelle Douala depuis OpenWeatherMap	Précipitations, température, humidité
GET	/api/meteo/previsions	Prévisions 6h pour Douala	Prévisions heure par heure avec probabilité pluie

5. Jobs automatiques (Cron Jobs)

Le backend tourne en permanence et exécute ces tâches planifiées automatiquement :

Job	Fréquence	Ce qu'il fait	Sortie
weather-check	Toutes les 30 min	Appel OpenWeatherMap /forecast + /weather. Si précipitations > seuil Gemini → génère alertes préventives et actives	Alertes écrites dans PostgreSQL + Firestore
scraper-onacc	Toutes les heures	Puppeteer ouvre onacc.cm, extrait les bulletins d'alerte officiels, parse région + niveau + date	Signalements bruts stockés dans PostgreSQL
scraper-social	Toutes les 15 min	Scrape Twitter/X et posts Facebook publics sur les mots clés : 'inondation Douala', 'flood Douala', 'eau Ndokotti', 'route bloquée Bépanda'	Textes bruts stockés pour analyse Gemini
gemini-analysis	À chaque nouveau signalement brut	Envoie le texte brut à Gemini API pour extraction : quartier, niveau, crédibilité, type (nouvelle alerte ou confirmation)	Alerte structurée publiée sur la carte si crédible
auto-resolve	Toutes les heures	Vérifie toutes les alertes actives. Si dernière confirmation > 3h OU plus de pluie forte selon météo → statut = résolu	Alertes obsolètes retirées de la carte
sync-firestore	À chaque modification	Synchronise les alertes actives de PostgreSQL vers Firebase	Firestore à jour, Angular notifié via

Job	Fréquence	Ce qu'il fait	Sortie
	PostgreSQL	Firestore pour le temps réel Angular	onSnapshot

6. Rôle de l'IA — Gemini API

Gemini est utilisé côté backend pour trois fonctions distinctes :

6.1 Analyse des signalements texte

Chaque signalement brut (citoyen ou scrapé) est envoyé à Gemini pour extraction structurée :

```
// Input envoyé à Gemini :
"l'eau monte vite ici à Ndokotti c'est dangereux"

// Prompt système :
"Tu es un analyseur de signalements d'inondation à Douala, Cameroun.
Extrais : quartier (parmi la liste officielle), niveau (léger/moyen/dangereux),
crédibilité (true/false), type (nouvelle_alerte/confirmation).
Réponds uniquement en JSON."

// Output Gemini :
{
  quartier: 'Ndokotti',
  niveau: 'dangereux',
  credible: true,
  type: 'nouvelle_alerte',
  resume: 'Montée rapide des eaux signalée à Ndokotti'
}
```

6.2 Seuil de déclenchement intelligent

Au lieu d'un seuil fixe (20mm/h pour tous les quartiers), Gemini calcule un seuil adaptatif par quartier selon l'historique :

```
// Input envoyé à Gemini :
{
  quartier: 'Ndokotti',
  historique_alertes: [...], // 90 derniers jours
  saison: 'pluies_mars_juin',
}
```

```

    meteo_actuelle: { precipitation: 15, humidity: 92 }
  }

// Output Gemini :
{
  seuil_mm_h: 12,           // Ndokotti inonde dès 12mm/h
  probabilite_inondation: 87, // % de risque
  recommandation: 'Alerte préventive recommandée'
}

```

6.3 Résumé intelligent pour la carte

Gemini génère un résumé lisible pour chaque alerte active, affiché dans la fiche sur la carte :

```

// Output affiché dans l'app :
"Ndokotti inondé depuis 2h. 8 confirmations citoyennes.
Évitez la rue Mandessi Bell.
Résolution estimée dans 1h30 si la pluie cesse."

```

7. Modèle de données PostgreSQL

Table : zones_a_risque (données statiques)

Champ	Type	Description
id	UUID PRIMARY KEY	Identifiant unique
nom_quartier	VARCHAR(100)	Ex: Ndokotti, Bépanda, Makepe, Logpom, PK8-PK14, Ndog-Bong, Melen
lat	DECIMAL(10,7)	Latitude du centroïde du quartier
lng	DECIMAL(10,7)	Longitude du centroïde du quartier
polygone	GEOMETRY(POLYGON)	Contour géospatial du quartier (PostGIS)
niveau_risque_base	ENUM(faible, moyen, élevé)	Niveau de risque de base issu de l'ONACC
seuil_mm_h	INTEGER	Seuil de précipitations déclencheur (mis à jour par Gemini)
source	VARCHAR(50)	ONACC ou historique_terrain

Table : alertes

Champ	Type	Description
id	UUID PRIMARY KEY	Identifiant unique
type	ENUM(préventive, active)	Préventive = risque détecté. Active = inondation en cours
source	ENUM(auto, citoyen, scraping_onacc, scraping_social)	Origine de l'alerte
zone_id	UUID FK → zones_a_risque	Zone concernée
niveau	ENUM(léger, moyen, dangereux)	Niveau de danger
heure_debut	TIMESTAMP	Heure de création de l'alerte
heure_prevue	TIMESTAMP NULL	Heure prévue d'inondation (alertes préventives uniquement)
nb_confirmations	INTEGER DEFAULT 0	Nombre de confirmations citoyennes
statut	ENUM(actif, résolu, en_résolution)	État actuel de l'alerte
resume_gemini	TEXT	Résumé lisible généré par Gemini
photo_url	VARCHAR(500) NULL	URL Firebase Storage si photo jointe
firestore_id	VARCHAR(100)	ID Firestore correspondant pour synchronisation temps réel

Table : signalements_bruts

Champ	Type	Description
id	UUID PRIMARY KEY	Identifiant unique
texte_brut	TEXT	Contenu brut scrapé ou saisi par le citoyen
source	VARCHAR(100)	twitter, facebook, onacc, citoyen
analyse_gemini	JSONB NULL	Résultat JSON de l'analyse Gemini
alerte_generee_id	UUID NULL FK → alertes	Alerte générée si signalement validé
created_at	TIMESTAMP	Date de collecte

Table : confirmations

Champ	Type	Description
id	UUID PRIMARY KEY	Identifiant unique
alerte_id	UUID FK → alertes	Alerte confirmée
user_uid	VARCHAR(128)	UID Firebase Auth du citoyen
type	ENUM(confirme, résolu)	Confirme que c'est encore là, ou signale que c'est passé
created_at	TIMESTAMP	Heure de la confirmation

8. Règles métier

Règle	Détail
Seuil de déclenchement	Précipitations > seuil_mm_h de la zone (calculé par Gemini, par défaut 20mm/h)
Validation signalement citoyen	Un signalement citoyen nécessite au moins 2 confirmations avant d'apparaître sur la carte
Anti-doublon confirmation	Un citoyen ne peut pas confirmer sa propre alerte (vérification via user_uid)
Résolution automatique	Statut → résolu si dernière confirmation > 3h OU météo : plus de pluie forte
Résolution manuelle	Un citoyen connecté peut marquer une alerte comme résolue
Alerte préventive → active	Une alerte préventive passe en active quand l'heure_prevue est atteinte ET pluie confirmée
Priorité des sources	Signalement ONACC officiel > Détection auto météo > Signalement citoyen > Scraping social

9. Synchronisation PostgreSQL ↔ Firestore

Angular utilise Firebase Firestore pour recevoir les alertes en temps réel sans polling. Voici comment la synchronisation fonctionne :

- ```
1. Backend écrit/met à jour une alerte dans PostgreSQL
 |
 ▼
2. Backend écrit la même alerte dans Firestore (collection 'alertes')
```

avec les champs essentiels : id, zone, niveau, statut, coordonnées GPS

↓



3. Angular est abonné via onSnapshot :

```
firestore.collection('alertes').where('statut', '==', 'actif')
```

↓



4. Angular reçoit la mise à jour instantanément

→ Nouveau marqueur sur Google Maps sans rechargement de page

*PostgreSQL est la source de vérité principale. Firestore est uniquement un canal de diffusion temps réel vers Angular. En cas de divergence, PostgreSQL prime.*

## 10. Variables d'environnement requises

| Variable                 | Description                         | Obtention                                                    |
|--------------------------|-------------------------------------|--------------------------------------------------------------|
| OPENWEATHER_API_KEY      | Clé API OpenWeatherMap              | openweathermap.org — compte gratuit                          |
| GEMINI_API_KEY           | Clé API Gemini                      | aistudio.google.com — compte Google gratuit                  |
| DATABASE_URL             | URL de connexion PostgreSQL         | Railway / Render lors du déploiement                         |
| FIREBASE_SERVICE_ACCOUNT | JSON credentials Firebase Admin SDK | Console Firebase → Paramètres du projet → Comptes de service |
| PORT                     | Port d'écoute du serveur Express    | Par défaut 3000                                              |
| FRONTEND_URL             | URL du frontend Angular             | Pour la configuration CORS                                   |

## 11. Déploiement recommandé

| Composant                  | Plateforme        | Pourquoi                                                                    | Coût                |
|----------------------------|-------------------|-----------------------------------------------------------------------------|---------------------|
| Backend Node.js            | Railway           | Déploiement en 1 clic depuis GitHub, PostgreSQL intégré, logs en temps réel | Gratuit (500h/mois) |
| Base de données PostgreSQL | Railway (intégré) | Inclus avec le backend Railway, pas de configuration séparée                | Inclus              |
| Frontend Angular           | Firebase Hosting  | Gratuit, CDN mondial, intégration                                           | Gratuit             |

| Composant         | Plateforme       | Pourquoi                                   | Coût    |
|-------------------|------------------|--------------------------------------------|---------|
|                   |                  | Firebase native                            |         |
| Fichiers / Photos | Firebase Storage | Déjà dans la stack, quota gratuit généreux | Gratuit |
| Auth utilisateurs | Firebase Auth    | Déjà dans la stack, OAuth Google inclus    | Gratuit |

## 12. Checklist de livraison backend

---

- API REST opérationnelle avec tous les endpoints de la section 4
- Cron jobs actifs : weather-check, scraper-onacc, scraper-social, gemini-analysis, auto-resolve
- Synchronisation PostgreSQL → Firestore fonctionnelle
- Gemini API intégrée pour l'analyse des signalements et les seuils intelligents
- Zones à risque de Douala préchargées dans PostgreSQL (Ndokotti, Bépanda, Makepe, Logpom, PK8-PK14, Ndog-Bong, Melen)
- Variables d'environnement documentées dans un fichier .env.example
- API déployée et accessible via une URL publique pour Angular
- Documentation Swagger / Postman des endpoints partagée avec le frontend